COMPILER DESIGN LAB 10 : BISON

Date	@March 31, 2026

Name: Devadathan N R

Reg No: 230905010

Roll No: 04

Class: CSE - A1

Question 1

To check a valid declaration statement.

CODE:

l10q1.l

```
%{
#include "l10q1.tab.h"
%}

%%
"int"|"float"|"char"|"double" { return TYPE; }
[a-zA-Z][a-zA-Z0-9_]* { return ID; }
","                           { return COMMA; }
";"                           { return SC; }
[ \t\n]                       { /* Skip whitespace */ }
.                             { return yytext[0]; }
%%

int yywrap() {
    return 1;
}
```

l10q1.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(char *s);
%}

%token TYPE ID COMMA SC

%%
```

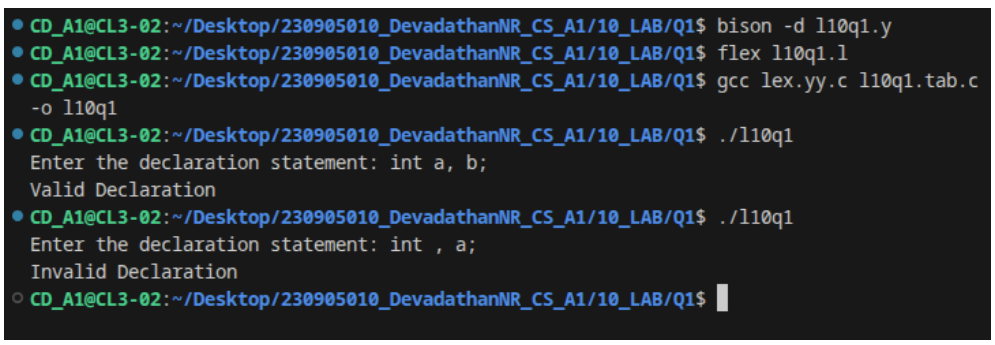
```
start: TYPE list SC { printf("Valid Declaration\n"); exit(0); }
     ;

list: ID
    | ID COMMA list
    ;
%%

void yyerror(char *msg) {
    printf("Invalid Declaration\n");
    exit(0);
}

int main() {
    printf("Enter the declaration statement: ");
    yyparse();
    return 0;
}
```

OUTPUT:



Question 2

To check a valid decision making statements.

CODE:

l10q2.l

```
%{
#include "l10q2.tab.h"
%}

%%
"if"                    { return IF; }
"else"                  { return ELSE; }
[a-zA-Z][a-zA-Z0-9_]* { return ID; }
```

```
[0-9]+                  { return NUM; }
"<="|">="|"=="|"!="      { return RELOP; }
[<>]                    { return RELOP; }
\n                      { return NL; }
[ \t]                   { /* skip whitespace */ }
.                       { return yytext[0]; }
%%

int yywrap() { return 1; }
```

l10q2.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(char *s);
%}

%token IF ELSE ID NUM RELOP NL

%%
start: if_stmt NL { printf("Valid Decision Making Statement\n"); exit(0); }
     ;

if_stmt: IF '(' cond ')' '{' body '}'
       | IF '(' cond ')' '{' body '}' ELSE '{' body '}'
       ;

cond: ID RELOP ID
    | ID RELOP NUM
    ;

body: ID ';'
    |
    ;
%%

void yyerror(char *s) {
    printf("Invalid Decision Making Statement\n");
    exit(0);
}

int main() {
    printf("Enter the if statement: ");
    yyparse();
    return 0;
}
```

OUTPUT:

```
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q2$ bison -d l10q2.y
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q2$ flex l10q2.l
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q2$ gcc lex.yy.c l10q2.tab.c
-o l10q2
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q2$ ./l10q2
Enter the if statement: if (a < 10) { x; }
Valid Decision Making Statement
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q2$ ./l10q2
Enter the if statement: if (a == 5) { } else { y; }
Valid Decision Making Statement
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q2$
```

Question 3

To evaluate an arithmetic expression involving operations +,-,* and /.

CODE:

l10q3.l

```
%{
#include "l10q3.tab.h"
extern int yylval;
%}

%%
[0-9]+          { yylval = atoi(yytext); return NUMBER; }
\n              { return NL; }
[ \t]           { /* skip whitespace */ }
.               { return yytext[0]; }
%%

int yywrap() { return 1; }
```

l10q3.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(char *s);
%}

%token NUMBER NL
/* Standard math precedence: * and / are higher than + and - */
%left '+' '-'
```

```
%left '*' '/'

%%
stmt: exp NL { printf("Result: %d\n", $1); exit(0); }
    ;

exp: exp '+' exp { $$ = $1 + $3; }
   | exp '-' exp { $$ = $1 - $3; }
   | exp '*' exp { $$ = $1 * $3; }
   | exp '/' exp {
        if($3 == 0) {
            printf("Error: Division by zero\n");
            exit(0);
        } else {
            $$ = $1 / $3;
        }
     }
   | '(' exp ')' { $$ = $2; }
   | NUMBER      { $$ = $1; }
   ;
%%

void yyerror(char *s) {
    printf("Invalid Expression\n");
    exit(0);
}

int main() {
    printf("Enter arithmetic expression: ");
    yyparse();
    return 0;
}
```

OUTPUT:

```
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q3$ bison -d l10q3.y
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q3$ flex l10q3.l
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q3$ gcc lex.yy.c l10q3.tab.c
-o l10q3
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q3$ ./l10q3
Enter arithmetic expression: 10 + 20 * 2
Result: 50
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q3$ ./l10q3
Enter arithmetic expression: (10 + 20) * 2
Result: 60
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q3$
```

Question 4

To validate a simple calculator using postfix notation. The grammar rules are as follows

```
input → input line | ε
line → '\n' | exp '\n'
exp → num | exp exp '+'
| exp exp '-'
| exp exp '*'
| exp exp '/'
| exp exp '^'
| exp 'n'
```

CODE:

l10q4.l

```
%{
#include "l10q4.tab.h"
extern int yylval;
%}

%%
[0-9]+  { yylval = atoi(yytext); return NUM; }
"+"     { return '+'; }
"-"     { return '-'; }
"*"     { return '*'; }
"/"     { return '/'; }
"n"     { return 'n'; }
\n      { return NL; }
[ \t]   { /* skip whitespace */ }
.       { return yytext[0]; }
%%

int yywrap() { return 1; }
```

l10q4.y

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex();
void yyerror(char *s);
%}

%token NUM NL

%%
start: exp NL { printf("Result: %d\n", $1); exit(0); }
     ;
```

```
exp: NUM           { $$ = $1; }
   | exp exp '+'   { $$ = $1 + $2; }
   | exp exp '-'   { $$ = $1 - $2; }
   | exp exp '*'   { $$ = $1 * $2; }
   | exp exp '/'   {
        if($2 == 0) {
            printf("Error: Division by zero\n");
            exit(0);
        } else {
            $$ = $1 / $2;
        }
     }
   | exp 'n'       { $$ = -$1; }
   ;
%%

void yyerror(char *s) {
    printf("Invalid Postfix Expression\n");
    exit(0);
}

int main() {
    printf("Enter postfix expression: ");
    yyparse();
    return 0;
}
```

OUTPUT:

```
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q4$ bison -d l10q4.y
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q4$ flex l10q4.l
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q4$ gcc lex.yy.c l10q4.tab.c
-o l10q4
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q4$ ./l10q4
Enter postfix expression: 5 3 +
Result: 8
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q4$ ./l10q4
Enter postfix expression: 10 5 * 2 +
Result: 52
CD_A1@CL3-02:~/Desktop/230905010_DevadathanNR_CS_A1/10_LAB/Q4$ ./l10q4
Enter postfix expression: 10 n
Result: -10
```